UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CS3014 - Estructuras de Datos Avanzadas
Árboles estáticos
Semana 12

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

28 de octubre de 2026

Objetivos de la sesión



Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Resolver RMQ estático con la Sparse Table y con bloques, y medir con el par $\langle P(n), Q(n) \rangle$
- Construir el cartesian tree y reducir RMQ a LCA, y LCA a ± 1 RMQ con el Euler tour

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Resolver RMQ estático con la Sparse Table y con bloques, y medir con el par $\langle P(n), Q(n) \rangle$
- Construir el cartesian tree y reducir RMQ a LCA, y LCA a ± 1 RMQ con el Euler tour
- Llegar a $\langle O(n), O(1) \rangle$ con el método de los cuatro rusos

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Dos preguntas sobre datos que no cambian

¿Cuál es el menor valor entre las posiciones i y j (RMQ)? ¿Cuál es el ancestro común más bajo de dos nodos (LCA)?

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Dos preguntas sobre datos que no cambian

¿Cuál es el menor valor entre las posiciones i y j (RMQ)? ¿Cuál es el ancestro común más bajo de dos nodos (LCA)?

Una sola caja de herramientas

Las dos se resuelven con **preprocesamiento** $O(n)$ **y consulta** $O(1)$, y se reducen una a la otra. Son la base de LCP en strings, de consultas sobre caminos y de muchos algoritmos que las usan como subrutina.

Range Minimum Query

Range Minimum Query

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Problema

Dado un arreglo $A[0..n-1]$ que se preprocesa, responder consultas

$$\mathrm{RMQ}_A(i,j) = \arg\min\{A[i], A[i+1], \ldots, A[j]\}, \qquad 0 \leq i \leq j < n$$

es decir, el **índice** $k \in [i,j]$ donde está el mínimo.

Dos observaciones

Devolver el índice es más general que devolver el valor: con k se obtiene $A[k]$. Y el arreglo **no cambia** (es *estático*): solo hay consultas.

¿Cómo medimos una solución?

Con el par $\langle P(n),\ Q(n)\rangle$: tiempo de preprocesamiento y tiempo por consulta.

Las primeras soluciones

Sin preprocesar: $\langle O(1),\ O(n)\rangle$

Recorrer $A[i..j]$ en cada consulta: 10^{12} operaciones con $n = 10^6$ y 10^6 consultas.

Tabla completa: $\langle O(n^2),\ O(1)\rangle$

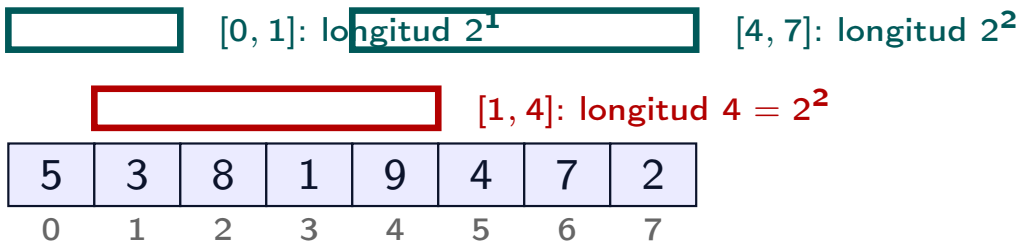
$M[i][j]$ para todo par, llenada con $M[i][j] =$ el mejor entre $M[i][j-1]$ y j. Pero n^2 celdas son inaceptables.

Segment tree: $\langle O(n),\ O(\lg n)\rangle$

Estándar, pero como el arreglo es estático pagamos por actualizaciones que nunca usamos.

Buscamos casi n de preprocesamiento y una consulta que no dependa de $j - i$.

La idea: potencias de 2



Una tabla solo con intervalos de longitud potencia de 2

Para cada inicio i y cada k con $i + 2^k \leq n$ guardamos

$$M[k][i] = \arg\min\{A[i], \ldots, A[i + 2^k - 1]\}$$

Hay n inicios y $\lfloor \lg n \rfloor + 1$ longitudes: $O(n \lg n)$ celdas.

Cómo se llena

Un intervalo de longitud 2^k son dos de longitud 2^{k-1}

$$[i,\ i+2^k-1] = [i,\ i+2^{k-1}-1] \cup [i+2^{k-1},\ i+2^k-1]$$

y el mínimo del intervalo grande es el mejor de los dos mínimos.

Recurrencia

$$M[0][i] = i, \qquad M[k][i] = \text{mejor entre } M[k-1][i] \text{ y } M[k-1][i+2^{k-1}]$$

Cada celda se calcula en $O(1)$ a partir del nivel anterior: el tiempo total es $O(n \lg n)$.

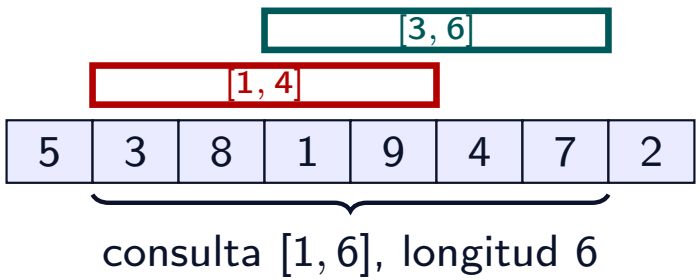
Un ejemplo

i	0	1	2	3	4	5	6	7
$A[i]$	5	3	8	1	9	4	7	2

$M[0][i]$	0	1	2	3	4	5	6	7
$M[1][i]$	1	1	3	3	5	5	7	
$M[2][i]$	3	3	3	3	7			
$M[3][i]$	3							

$M[1][2] = 3$ porque $A[3] = 1 < A[2] = 8$. $M[2][4] = 7$ compara $M[1][4] = 5$ $(A = 4)$ con $M[1][6] = 7$ $(A = 2)$.

La consulta



Dos intervalos que se solapan

Sea $k = \lfloor \lg(j - i + 1) \rfloor$. Entonces $2^k \leq j - i + 1 < 2^{k+1}$ y los intervalos $[i,\ i + 2^k - 1]$ y $[j - 2^k + 1,\ j]$ **cubren** $[i, j]$.

El mínimo de la unión

Es el mejor de los dos mínimos. Que se solapen no importa: el mínimo de un conjunto no cambia si un elemento se cuenta dos veces.

Consulta en $O(1)$

Algoritmo 1: RMQ-SPARSE(i, j)

$k \leftarrow \lfloor \lg(j - i + 1) \rfloor$
$a \leftarrow M[k][i]; \quad b \leftarrow M[k][j - 2^k + 1]$
si $A[a] \leq A[b]$ **entonces**
  devolver a
en otro caso
  devolver b

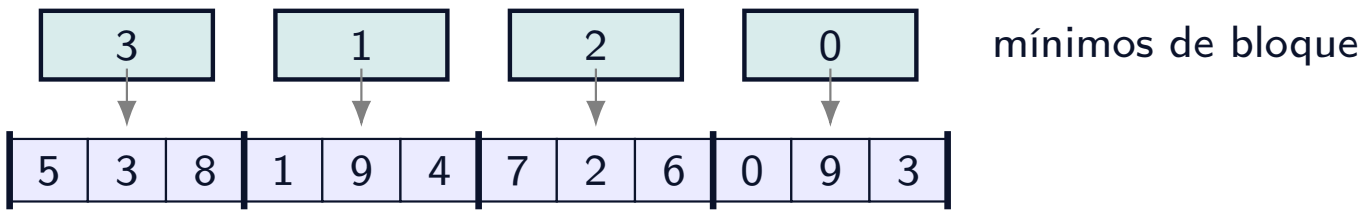
Detalles

$\lfloor \lg x \rfloor$ se obtiene en $O(1)$ con una tabla precalculada de tamaño n o con una instrucción de "contar ceros iniciales". La consulta hace 2 lecturas de tabla y 1 comparación.

Solo funciona con operaciones idempotentes

mín$(x, x) = x$. Para la **suma** esta técnica no sirve.

Partir el arreglo en bloques



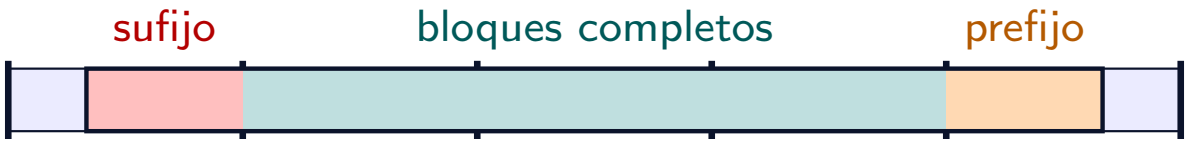
Bloques de tamaño b

Hay n/b bloques. Guardamos el mínimo de cada uno y construimos la Sparse Table **sobre esos n/b valores**.

Memoria de la Sparse Table

$O\left(\frac{n}{b} \lg \frac{n}{b}\right)$. Con $b = \lg n$ queda $O\left(\frac{n}{\lg n} \lg n\right) = O(n)$.

Una consulta atraviesa tres zonas



Consulta $[i, j]$ en bloques distintos

1. **Sufijo** del bloque de i.
2. **Bloques completos** entre los dos: Sparse Table sobre los mínimos de bloque, $O(1)$.
3. **Prefijo** del bloque de j.

Sufijos y prefijos

Se resuelven guardando, para cada posición, el mínimo desde el inicio de su bloque y hasta su final: $O(n)$ memoria y $O(1)$ por consulta.

Lo que queda abierto

Si i y j están en el *mismo* bloque

Ninguna de las tres zonas aplica: i y j son interiores a un bloque de tamaño $b = \lg n$.

Tres opciones y ninguna conviene

Recorrer el bloque cuesta $O(\lg n)$. Una Sparse Table por bloque vuelve a costar $O(n \lg \lg n)$. Una tabla completa por bloque cuesta $O(n \lg n)$.

Lo resolvemos más adelante en esta clase, después de ver LCA. Para ello nos conviene conocer la estructura de los problemas *dentro* de un bloque, y la herramienta es un árbol.

Cartesian Tree y LCA

Cartesian Tree

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

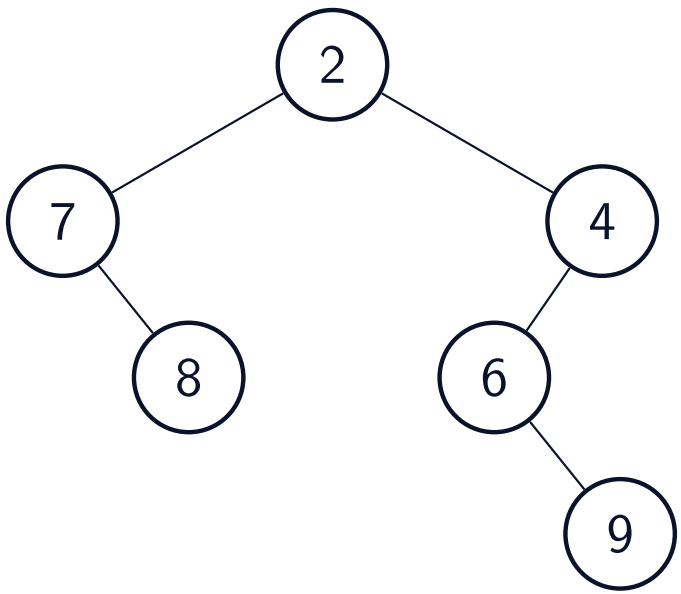
El cartesian tree T_A de un arreglo $A[0..n-1]$ es un árbol binario con n nodos (uno por posición) tal que

- la **raíz** es la posición m del mínimo de A;
- su subárbol izquierdo es el cartesian tree de $A[0..m-1]$ y su subárbol derecho el de $A[m+1..n-1]$.

Dos propiedades

Cumple la propiedad de **heap** (el padre es menor que sus hijos) y el recorrido **inorden** devuelve $0, 1, \ldots, n-1$.

Ejemplo: $A = [7,\ 8,\ 2,\ 6,\ 9,\ 4]$



La raíz es el mínimo (2, posición 2). A la izquierda queda $[7, 8]$ y a la derecha $[6, 9, 4]$, con raíz 4.

RMQ es un problema de LCA

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Teorema

Sea T_A el cartesian tree de A (con empates resueltos por la posición). Entonces

$$\mathrm{RMQ}_A(i,j) = \mathrm{LCA}_{T_A}(i,j).$$

Consecuencia

Si sabemos resolver LCA, sabemos resolver RMQ. Falta construir T_A en tiempo lineal.

Construcción lineal: una pila

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Se construye de izquierda a derecha

Se mantiene la **columna derecha** del árbol (de la raíz hacia abajo siguiendo hijos derechos) en una pila. Al llegar a $A[i]$:

- se desapilan los nodos v con $A[v] > A[i]$, y el **último** desapilado pasa a ser hijo izquierdo de i;
- i pasa a ser hijo derecho del nodo que queda en el tope (si hay) y se apila.

¿Por qué es correcto?

Los nodos desapilados son mayores que $A[i]$, así que quedan debajo de i (heap); como i llega a la derecha de todos, sus posiciones son menores: forman el subárbol izquierdo de i.

Construcción: pseudocódigo y costo

Algoritmo 2: ÁRBOL-CARTESIANO(A, n)

```
pila ← vacía;    izq[·] ← −1;    der[·] ← −1
para i ← 0 a n − 1 hacer
    ultimo ← −1
    mientras pila no vacía y A[tope(pila)] > A[i] hacer
        ultimo ← desapilar(pila)
    izq[i] ← ultimo
    si pila no vacía entonces der[tope(pila)] ← i
    apilar(pila, i)
devolver el fondo de la pila                          // la raíz
```

$O(n)$

Cada posición se apila una vez y se desapila a lo más una vez: el total de iteraciones del **while** es $\leq n$.

Lowest Common Ancestor

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Problema

Dado un árbol con raíz de n nodos que se preprocesa, responder consultas $\mathrm{LCA}(u, v)$: el nodo más profundo que es ancestro de u y de v (un nodo es ancestro de sí mismo).

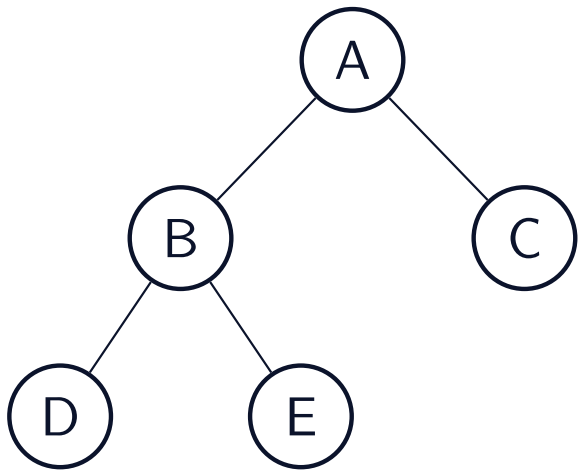
Para qué sirve

Distancia entre dos nodos, $d(u, v) = d(u) + d(v) - 2\,d(\mathrm{LCA})$; ¿es u ancestro de v?; el camino entre dos nodos; consultas sobre caminos.

La reducción también va en el otro sentido

Acabamos de reducir RMQ a LCA. Ahora reducimos LCA a un caso particular de RMQ, y ese caso es el que sabremos resolver en tiempo lineal.

Euler tour



Se anota el nodo cada vez que se pasa por él

Recorrido en profundidad: se escribe el nodo al entrar y cada vez que se regresa a él desde un hijo. Un árbol con n nodos da una lista E de $2n - 1$ entradas.

$E =$ A B D B E B A C A $\qquad D =$ 0 1 2 1 2 1 0 1 0 (profundidades)

LCA es un mínimo sobre profundidades

Teorema

Sea $first[v]$ la primera posición de v en E. Si $first[u] \leq first[v]$, entonces

$$\mathrm{LCA}(u, v) = E\left[\text{arg mín } D[first[u]..first[v]]\right].$$

Demostración

Sea $w = \mathrm{LCA}(u, v)$. Todo el tramo de E entre $first[u]$ y $first[v]$ ocurre dentro del recorrido de w, así que sus nodos están en el subárbol de w y tienen profundidad $\geq d(w)$. Además u y v están en subárboles de hijos distintos de w (o $u = w$): el recorrido tiene que **volver a pasar por** w entre ambos. Luego el mínimo de D en el tramo es $d(w)$ y lo alcanza w.

Ejemplo: $\mathrm{LCA}(D, E)$: $first[D] = 2$, $first[E] = 4$, $D[2..4] = 2, 1, 2$ con mínimo en $E[3] = \mathrm{B}$.

La propiedad ± 1

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Profundidades consecutivas

En D, dos entradas consecutivas **siempre** difieren en exactamente 1: en el recorrido se pasa de un nodo a su padre o a un hijo.

$$|D[t] - D[t-1]| = 1 \qquad \text{para todo } t$$

± 1 RMQ

Un RMQ sobre un arreglo con esta propiedad se llama ± 1 RMQ. Es un caso **particular** de RMQ: tiene mucha estructura.

Una pista

Un bloque de b entradas de un ± 1 RMQ tiene solo 2^{b-1} formas posibles. Con $b = \frac{1}{2} \lg n$ son $\sqrt{n}$ formas.

Ida y vuelta

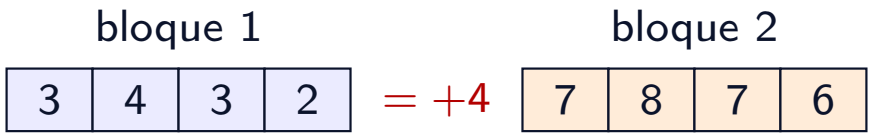


Conclusión

Si resolvemos ± 1 RMQ en $\langle O(n), O(1)\rangle$, entonces RMQ y LCA también. (Con la Sparse Table sobre el Euler tour ya tendríamos $\langle O(n \lg n), O(1)\rangle$.)

RMQ y LCA en tiempo lineal

Dos bloques que son "el mismo"



Misma forma, mismas respuestas

Los dos bloques difieren en una constante ($+4$): para todo rango $[l, r]$, el índice del mínimo es **el mismo** en ambos.

Lo que importa es el patrón de pasos

Restando el primer elemento, un bloque de un ± 1 RMQ queda determinado por su secuencia de pasos: $+, -, -$ en el ejemplo. Con b elementos hay $b - 1$ pasos y 2^{b-1} patrones.

¿Cuántos patrones distintos hay?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Elegimos $b = \frac{1}{2} \lg n$

$$2^{b-1} \leq 2^{\frac{1}{2} \lg n} = \sqrt{n}$$

Hay a lo más $\sqrt{n}$ patrones distintos, **sin importar n ni los valores**, aunque haya $\frac{n}{b} \gg \sqrt{n}$ bloques.

Entonces

Muchísimos bloques comparten patrón. Basta una tabla de respuestas **por patrón**, no por bloque.

La tabla por patrón

Para cada patrón p y cada par (l, r) con $0 \leq l \leq r < b$

$T[p][l][r]$ = posición del mínimo en el rango $[l, r]$ de un bloque con patrón p

Tamaño y tiempo

Hay $\leq \sqrt{n}$ patrones, $\binom{b}{2} + b \leq b^2$ pares:

$$\sqrt{n} \cdot b^2 = \sqrt{n} \cdot \tfrac{1}{4} \lg^2 n = O(\sqrt{n} \lg^2 n) = o(n)$$

Cada tabla se llena en $O(b^2)$ recorriendo r y manteniendo el mínimo; en total $o(n)$.

Preprocesamiento de ± 1 RMQ

Algoritmo 3: PREPROCESAR-PM1(D, n)

$b \leftarrow \max(1, \lfloor \lg n/2 \rfloor)$; $\quad nb \leftarrow \lceil n/b \rceil$
para *cada bloque* $t = 0, \ldots, nb - 1$ **hacer**
  $bmin[t] \leftarrow$ valor mínimo del bloque; $\quad bidx[t] \leftarrow$ su posición
  $pat[t] \leftarrow$ patrón de pasos del bloque (entero de $b - 1$ bits)
construir la Sparse Table sobre $bmin[0..nb - 1]$
para *cada patrón p que aparece* **hacer**
  llenar $T[p][l][r]$ para todo $0 \leq l \leq r < b$

Costo total

$O(n)$ (bloques) $+ O(\frac{n}{b} \lg \frac{n}{b}) = O(n)$ (macro) $+ o(n)$ (tablas): $O(n)$.

Consulta de ± 1 RMQ

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Algoritmo 4: $\mathrm{RMQ\text{-}PM1}(i, j)$

$s \leftarrow \lfloor i/b \rfloor$; $\quad t \leftarrow \lfloor j/b \rfloor$
si $s = t$ **entonces devolver** $s \cdot b + T[pat[s]][i \bmod b][j \bmod b]$
$a \leftarrow s \cdot b + T[pat[s]][i \bmod b][b-1]$ `// sufijo`
$c \leftarrow t \cdot b + T[pat[t]][0][j \bmod b]$ `// prefijo`
$m \leftarrow$ mín por valor entre a y c
si $t - s > 1$ **entonces**
  $u \leftarrow bidx[\mathrm{RMQ\text{-}Sparse}(s+1, t-1)]$ `// macro`
  $m \leftarrow$ mín por valor entre m y u
devolver m

Una consulta, paso a paso

$$D = [\,2, 3, 2, 1 \,|\, 2, 3, 4, 3\,], \quad b = 4, \quad \text{consulta } (i, j) = (1, 6)$$

Bloques

Bloque 0: patrón $+, -, -$. Bloque 1: patrón $+, +, -$.

Las dos partes micro

Sufijo del bloque 0 desde 1: $T[+--][1][3] = 3$, valor $D[3] = 1$. Prefijo del bloque 1 hasta j mód $4 = 2$: $T[++-][0][2] = 0$, es decir la posición 4, valor 2.

Respuesta

No hay bloques intermedios. mín$(1, 2) = 1$ en la posición 3.

El método de los cuatro rusos

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La técnica

Cuando el problema se divide en subproblemas **tan pequeños** que solo hay pocos tipos distintos, se **precalculan todas** las respuestas posibles y se consultan por índice.

Aquí

Un "subproblema" es un bloque de tamaño $\frac{1}{2} \lg n$. Su tipo es su patrón: $\sqrt{n}$ tipos. Las respuestas se precalculan una vez por tipo, no una vez por bloque.

El nombre viene de Arlazarov, Dinic, Kronrod y Faradžev (1970), que la usaron para multiplicar matrices booleanas.

Teorema: ± 1 RMQ en $\langle O(n), O(1) \rangle$

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

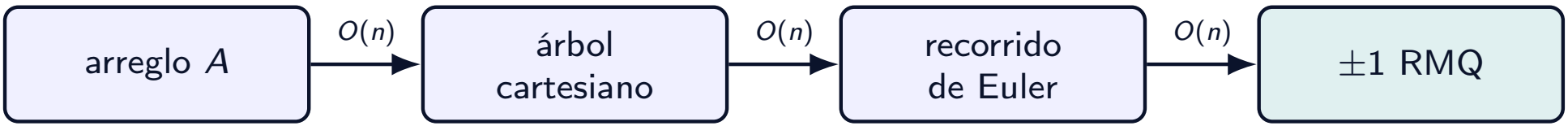
Resumen del algoritmo

- Macro: Sparse Table sobre los mínimos de bloque, $O(n)$.
- Micro: una tabla por patrón de bloque, $o(n)$.
- Consulta: a lo más 2 consultas micro y 1 macro, $O(1)$.

Cuentas

Componente	Memoria y tiempo	Consulta
Mínimos de bloque + patrones	$O(n)$	—
Sparse Table macro	$O(\frac{n}{b} \lg \frac{n}{b}) = O(n)$	$O(1)$
Tablas micro	$O(\sqrt{n} \lg^2 n)$	$O(1)$

LCA y RMQ general en $\langle O(n), O(1) \rangle$



La cadena de reducciones

$$\mathrm{RMQ}_A(i,j) = \mathrm{LCA}_{T_A}(i,j) = E\left[\mathrm{RMQ\text{-}PM1}(\mathit{first}[i], \mathit{first}[j])\right].$$

Resultado

Reducciones lineales y consulta $O(1)$: **RMQ y LCA en** $\langle O(n),\ O(1) \rangle$. Para LCA basta el Euler tour ($2n-1$ entradas) y $\mathrm{RMQ\text{-}PM1}$. (Harel y Tarjan, 1984; simplificado por Bender y Farach-Colton, 2000.)

Práctica

Práctica

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Problema	**Idea**
CSES 1647 — Static Range Minimum Queries	Sparse Table
Library Checker — Static RMQ	versión lineal con bloques
CSES 1688 — Company Queries II	Euler + RMQ (LCA)
CSES 1135 — Distance Queries	$d(u) + d(v) - 2\,d(\mathrm{LCA})$
CSES 1136 — Counting Paths	LCA + diferencias sobre el árbol
Library Checker — Lowest Common Ancestor	Euler tour + RMQ lineal

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- La Sparse Table cubre un intervalo con dos de longitud potencia de 2: $\langle n \lg n, 1 \rangle$, solo para operaciones idempotentes.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- La Sparse Table cubre un intervalo con dos de longitud potencia de 2: $\langle n \lg n, 1 \rangle$, solo para operaciones idempotentes.
- El cartesian tree se construye en $O(n)$ con una pila y cumple $\mathrm{RMQ} = \mathrm{LCA}$; el Euler tour reduce LCA a ± 1 RMQ.
- Un bloque de ± 1 RMQ tiene $\leq \sqrt{n}$ patrones: tablas por patrón y Sparse Table macro dan $\langle O(n), O(1) \rangle$.

Logros de la sesión



Logros de la sesión

- Sabemos construir y consultar una Sparse Table
- Sabemos construir un cartesian tree en tiempo lineal

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos construir y consultar una Sparse Table
- Sabemos construir un cartesian tree en tiempo lineal
- Sabemos reducir RMQ a LCA y LCA a ± 1 RMQ
- Sabemos resolver ± 1 RMQ en $\langle O(n), O(1) \rangle$

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias

Referencias

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Bibliografía citada

- Bender, M. A., & Farach-Colton, M. (2000). The LCA problem revisited. *LATIN'00*, 88–94.
- Fischer, J., & Heun, V. (2011). Space-efficient preprocessing schemes for range minimum queries on static arrays. *SIAM J. Comput.*, 40(2), 465–492.
- Demaine, Erik (2021). *6.851: Advanced Data Structures (Spring'21)*. `https://courses.csail.mit.edu/6.851/spring21/`.

Agradecimientos

Estas diapositivas están basadas en las clases de Luciano A. Romero Calla para este curso (a su vez basadas en las clases de Erik Demaine, MIT 6.851), así como en clases similares de otras universidades.
Lista de créditos: Luciano A. Romero Calla, Erik Demaine, Keith Schwarz.